ITEC Rio Cuarto
Carrera: Técnico Superior en Desarrollo de Software
Unidad Curricular: Frontend
Docente: Valentino Arballo

Examen Final Integrador

OBJETIVO:

Desarrollar una aplicación web en React que consuma información de una API de forma asíncrona, aplique enrutamiento con múltiples vistas y gestione el estado global de la aplicación.

MODALIDAD:

El examen tendrá una nota en conjunto con la materia ***de backend*** y nacerá de un promedio entre la nota particular de este ***EFI*** en conjunto con la nota particular del ***EFI*** de Python.

El desarrollo será en grupos con un **máximo** de hasta 2 personas. Este grupo debe ser el mismo entre los exámenes de ambas materias.

El examen se debe defender de manera oral, mostrando el proyecto en funcionamiento y respondiendo preguntas sobre el mismo.

Se usarán las horas de clase para el desarrollo del proyecto, quedando yo a disposición para cualquier consulta que surja durante el desarrollo del mismo.

La fecha límite de entrega para ambos proyectos será el **miércoles 10 de noviembre**.

Requisitos

- **R1: Enrutamiento y Navegación (react-router-dom)**
  - **Rutas Base:** Implementar al menos 6 rutas principales en la aplicación (**ejemplo**: Inicio /, Catálogo /items, Favoritos /favoritos)
  - **Rutas Dinámicas:** Definir al menos 2 rutas dinámica que reciba un parámetro para consultar y mostrar el detalle de un ítem en específico (**ejemplo**: /item/:id)
  - **Ruta Anidada:** Implementar al menos 1 ruta anidada utilizando subrutas (**ejemplo**: /items/categoria dentro de /items, o /perfil/ajustes)



- **R2: Consumo de API y Manejo de Estado (useState y useEffect)**
    - Consumir una API externa mediante peticiones asíncronas (fetch o axios) para **leer**, **crear** y **eliminar** datos.
    - Hacer uso obligatorio de useState para gestionar el estado de los datos, indicadores de carga (*loading*) y errores
    - Hacer uso obligatorio de useEffect para desencadenar las peticiones a la API en la carga inicial o ante cambios de filtros
    - Controlar los formularios y validar la carga del usuario (campos obligatorios, formato, longitud del input)
- **R3: Gestión de Estado Global con Context API**
    - Implementar Context API (createContext y useContext) para compartir y almacenar información global en la aplicación
    - *Ejemplos de uso:* Guardar una lista de "Favoritos" / "Carrito", selector de tema (Modo Claro / Oscuro)
- **R4: Hosting de la aplicación en Vercel**
    - Configuración de las variables de entorno
    - Poder entregar y mostrar la aplicación desplegada el día del examen.

Condiciones de entrega

- Documentar el desarrollo de tareas usando alguna herramienta de metodología ágil (como por ejemplo un tablero kanban)
- Separar la lógica en componentes modulares y reutilizables.
- Aplicar estilos con Tailwind.
- Modificar el archivo README.md agregando:
    - Título de la aplicación.
    - Breve descripción que explique la temática elegida y la API utilizada.
    - Pasos para ejecutar la aplicación de forma local (npm install, npm run dev / npm start).
    - Nombres y Apellidos de los integrantes.
- Entregar el enlace al repositorio público mediante **GitHub**.
- Usar ramas para organizar el desarrollo que sigan la estrategia GitFlow (main, develop, feature branches)